```
> ./run_diagnostic_playbook.sh
```

Google L4 DSA: The 24-Hour Cognitive Playbook

Visual recognition + core technique + mental templates for the 45-minute coding interview.

How to use this: Look at the visual → say the pattern out loud → recall the invariant → recall the template.

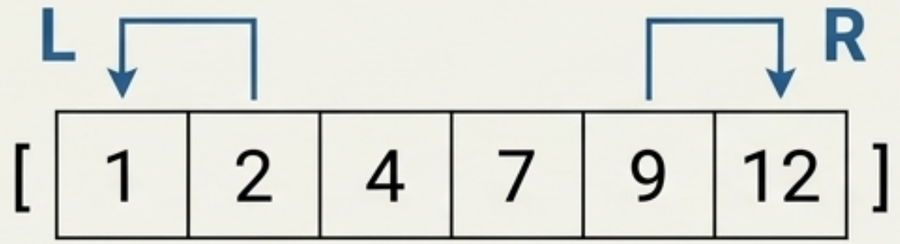
Gemini Notebook

THE DIAGNOSTIC MATRIX

Symptom (Problem feels like...)	**Diagnosis** (Think…)	**Prescription** (Core move)
Pair / opposite ends / sorted array	Two Pointers	Move L/R based on condition
Longest / shortest contiguous range	Sliding Window	Expand R, shrink L
Next greater / smaller / unresolved	Monotonic Stack	Pop while invariant breaks
Sorted / monotonic answer space	Binary Search	Discard half
Min/Max X where feasibility flips	BS on Answer	Binary-search X + can(X)
Level-by-level / minimum edges	BFS	Queue + visited
Explore all reachable nodes	DFS	Recursive or explicit stack
Dependencies / prerequisites	Topological Sort	Indegree or DFS states
Repeated smallest/largest	Heap	Keep best K / next candidate
All combinations / permutations	Backtracking	Choose → recurse → undo
Optimal result from overlapping	DP	State + transition

Gemini Notebook

Two Pointers

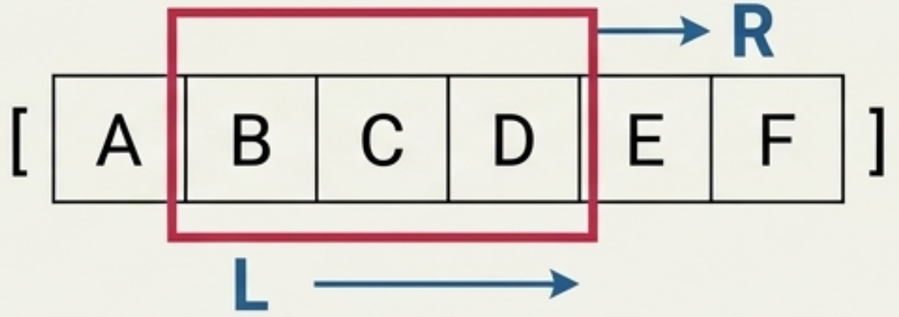


Recognize: sorted array + pair/target; palindrome; in-place compaction; opposite-end optimization.

Invariant: Every discarded region cannot contain a better/valid answer under the problem's ordering.

```
function twoSum(nums, target) {
  let left = 0, right = nums.length - 1;
  while (left < right) {
    let sum = nums[left] + nums[right];
    if (sum === target) return [left, right];
      return ...
    else if (sum < target) left++;
    else right--;
  }
}
```

Sliding Window

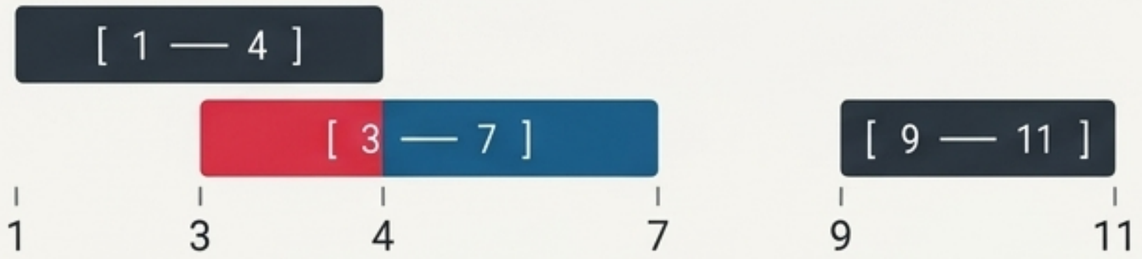


Recognize: longest/shortest substring/subarray; at most K; frequency constraints; contiguous range.

```
let left = 0;
for (let right = 0; right < s.length; right++) {
  // Expand window ...
  while (/* invalid condition */) {
    left++;
  } // Shrink window
  // Update result ...
}
```

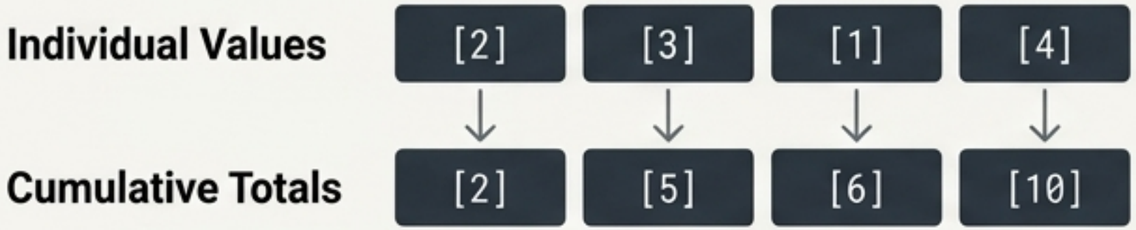
Gemini Notebook

Intervals



Core Move: Sort by start when relationships between intervals matter. Merge when the next start is within the current end.

Prefix Sums



Recognize: Repeated range sums or subarray-sum conditions.

Mental Model: Build cumulative totals so a range can be answered instantly from two prefix values.

```
function mergeIntervals(intervals) {
  if (intervals.length === 0) return [];
  intervals.sort((a, b) => a[0] - b[0]);

  const out = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const [start, end] = intervals[i];
    const currentEnd = out[out.length - 1][1];
    if (start <= currentEnd) {
      out[out.length - 1][1] = Math.max(currentEnd, end);
    } else {
      out.push([start, end]);
    }
  }
  return out;
}
```

Gemini Notebook

Binary Search: Search Space Reduction



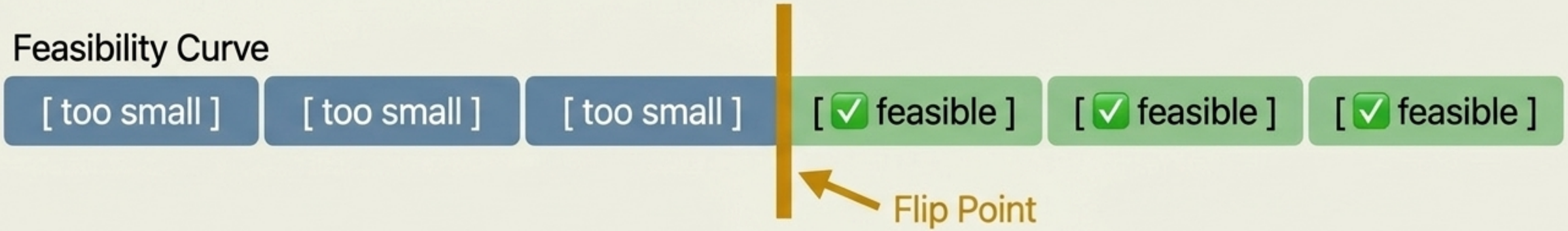
Diagnosis: Sorted data, first/last valid position, rotated sorted array, or a monotonic predicate.

Core Move: Ordered candidates. At each step, eliminate one half.

```
function binarySearch(nums, target) {
    let left = 0, right = nums.length - 1;
    while (left <= right) {
        let mid = Math.floor((left + right) / 2);
        if (nums[mid] === target) return mid;
        else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Gemini Notebook

Binary Search on Answer



The Invariant: The values themselves may not be sorted; the FEASIBILITY of an answer is monotonic.

Cognitive Trigger: "If X works, will every larger (or smaller) X also work?" If yes, search X.

Cognitive Trigger: "If X works, will every larger (or smaller) X also work?" If yes, search X.

```
function can(x) {
  // return true if x is feasible
}
```

```
while (lo < hi) {
  let mid = lo + Math.floor((hi - lo) / 2);
  if (can(mid)) hi = mid; else lo = mid + 1;
}
```

Gemini Notebook

Advanced Linear State: Monotonic Stack



Diagnosis: Next greater/smaller; previous greater/smaller; temperatures; histogram; nearest boundary.

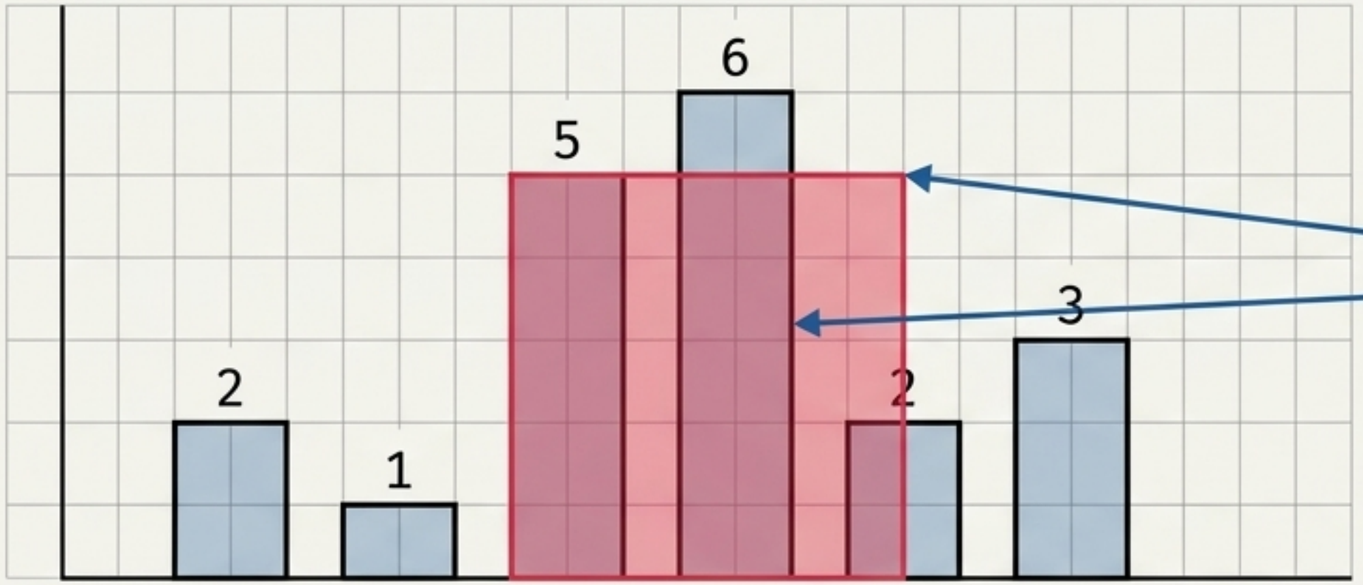
Core Mechanics: Keep unresolved candidates in monotonic order. When a new value resolves earlier candidates, pop them.

```
function dailyTemperatures(temperatures) {
    const n = temperatures.length;
    const result = new Array(n).fill(0);
    const stack = []; // Stores indices

    for (let i = 0; i < n; i++) {
        while (stack.length && temperatures[stack[stack.length - 1]] < temperatures[i]) {
            const j = stack.pop();                    // nums[i] resolves j
            result[j] = i - j;
        }
        stack.push(i);
    }
    return result;
}
```

Gemini Notebook

LC 84 Deep Dive: Largest Rectangle in Histogram



Popping height 5 reveals Right Boundary (current index).
New Stack Top reveals Left Boundary.

Core Recognition: “Largest rectangle / nearest smaller boundary” → monotonic increasing stack.

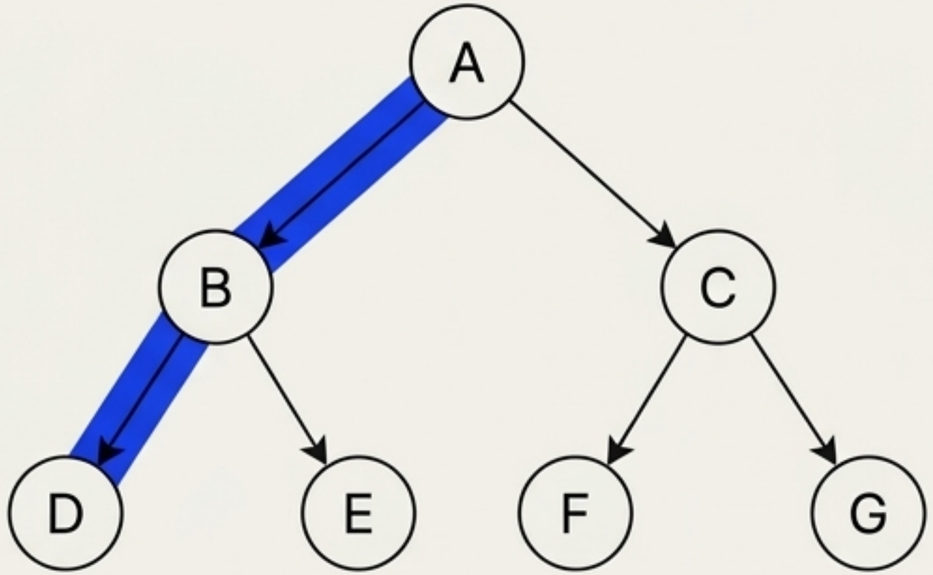
Area = height × width where width = i − left − 1

Interview Check: Be able to explain why the width is i − left − 1 and why a sentinel 0 appended to the end flushes the remaining stack.

Gemini Notebook

Traversal Architecture: DFS vs BFS

DFS (Depth-First Search)

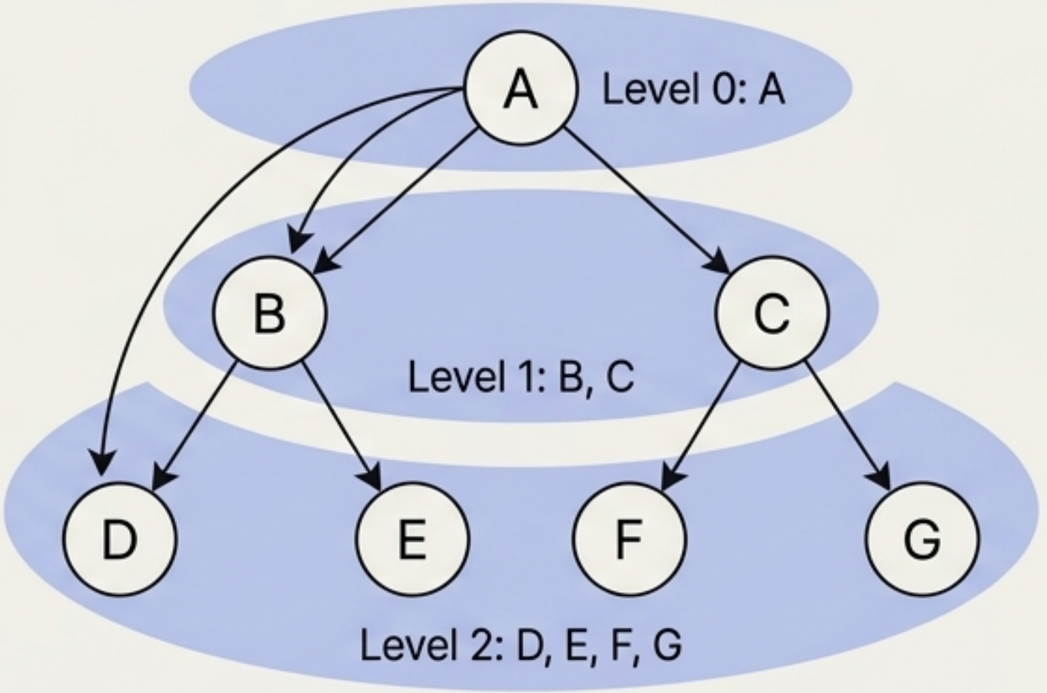


Triggers: Depth, height, path, subtree problems.

Mental Model: Go deep. Recursively ask: 'What should this subtree return to its parent?'

Mechanics: Recursive **stack** or explicit `stack.pop()`.

BFS (Breadth-First Search)

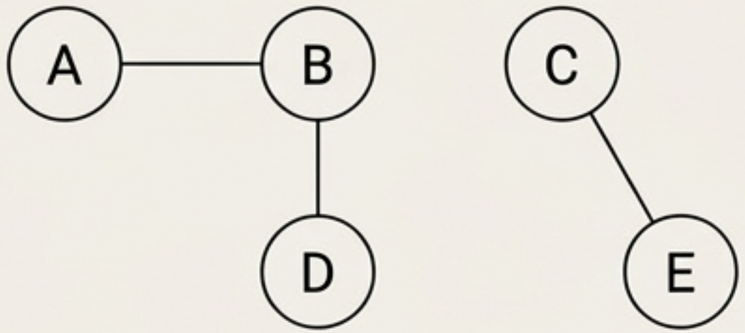


Triggers: Level-by-level, minimum edges, shortest path in unweighted graphs.

Mechanics: Queue.
JS Warning: Avoid `Array.shift()` (O(n)).
Use a head index: `const node = queue[head++];`

Gemini Notebook

Graph Basics



Rule: Nodes + edges + visited. The exact same DFS/BFS mechanics apply to adjacency lists and matrix grids.

Grid Specs: 4-direction neighbors + bounds check + visited set.

Topological Sort (Kahn's Algorithm)

[Indegree Array] → [Queue of 0-Indegrees] → [Pop & Decrement Neighbors]

Triggers: Prerequisites, dependencies, strict ordering.

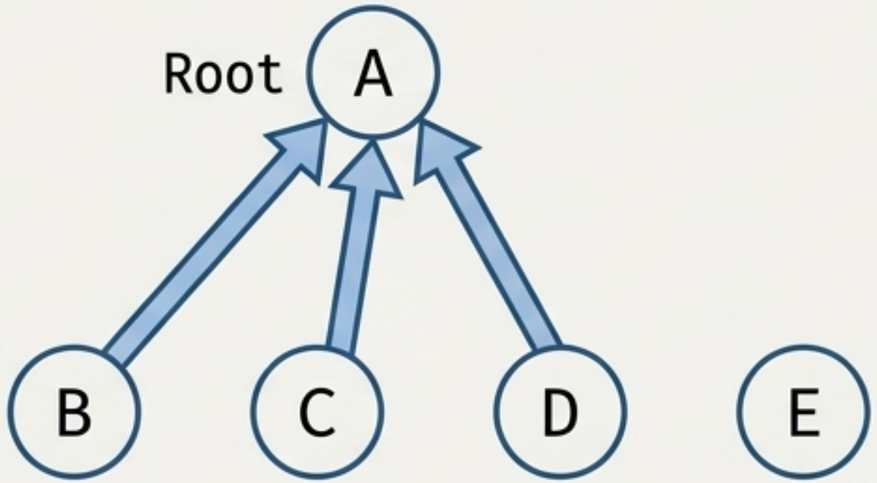
```
function topologicalSort(n, edges) {
  const indegree = Array(n).fill(0);
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adj[u].push(v);
    indegree[v]++;
  }
  const q = []; for (let i = 0; i < n; i++) {
    if (indegree[i] === 0) q.push(i);
    const u = q.shift();
      result.push(u);
    count++;
      for (const v of adj[u]) {
        if (--indegree[v] ==0) q.push(v);
      }
    }
  }
  return count < n ? [] : result;
  }
```

Dependency Check: If indegree becomes 0, add to queue.

Cycle Detection: count < n => cycle.

Gemini Notebook

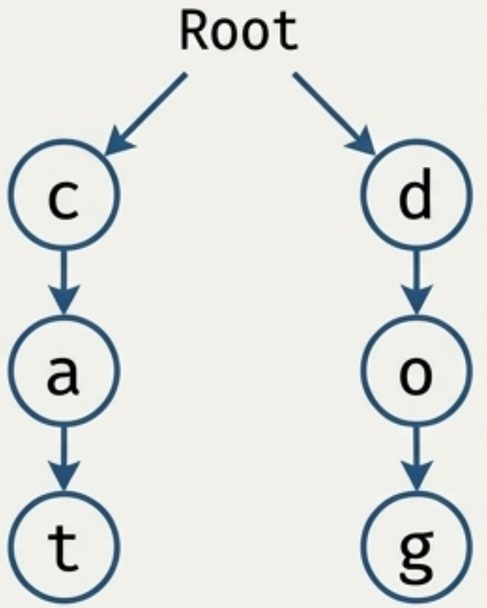
Union-Find (DSU)



Triggers: Dynamically joining components, connectivity after many unions, cycle detection in undirected graphs.

Must Know: Parent array + path compression + union by size/rank. Find follows parents; union connects components.

Trie (Prefix Tree)

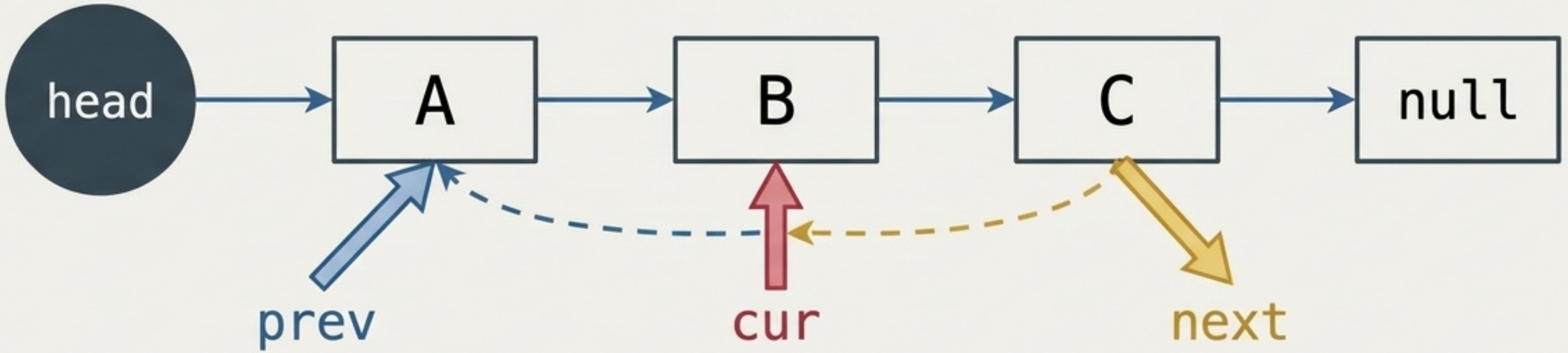


Triggers: Prefix, startsWith, dictionary words, autocomplete.

Mental Model: Walk one character per level. Excellent for prefix lookups. Do not over-invest in advanced variants for a 45-min interview.

Gemini Notebook

Linked Lists: Safe Pointer Manipulation



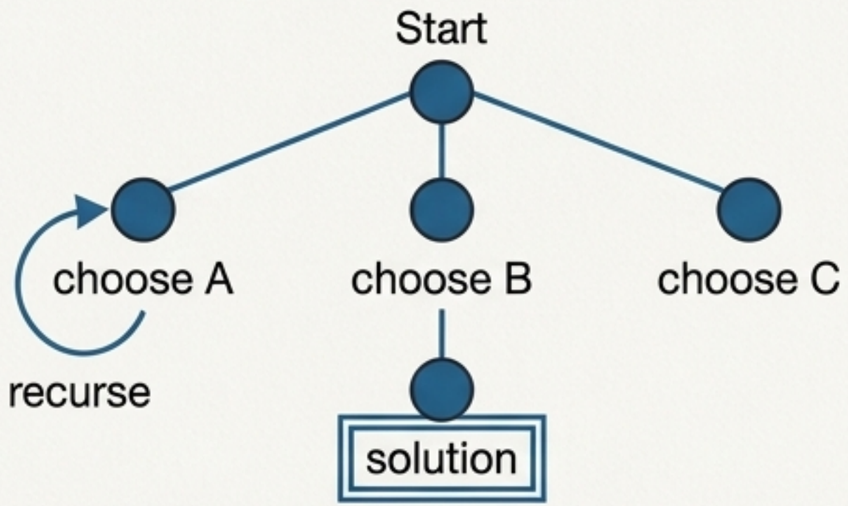
- **Core Mechanics:** Pure pointer manipulation.
- **Reusable Toolbelt:** Dummy node, prev/current/next sliding window, slow/fast pointers.
- **High-Value Set:** Reverse, detect cycle, find middle, merge two sorted lists, remove Nth from end.

```
1. const next = cur.next;
2. cur.next = prev;
3. prev = cur;
4. cur = next;
```

Gemini Notebook

Backtracking & Heaps / Top-K

Backtracking

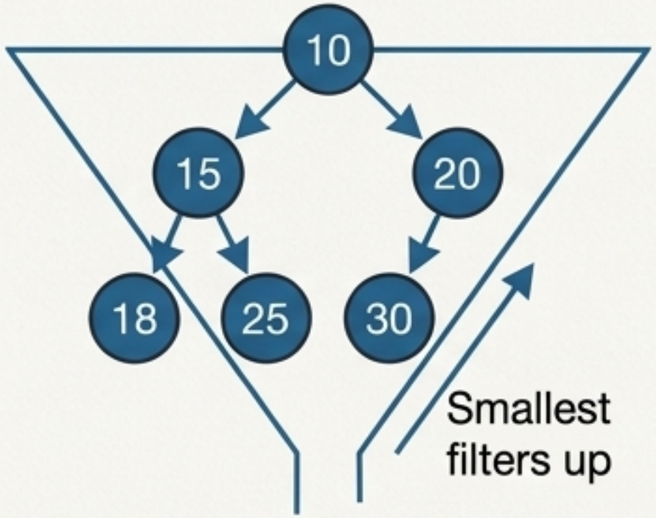


Triggers: All combinations / permutations. Explore a decision tree.

Core Pattern: Choose → recurse → undo.

```
path.push(choice);
backtrack(path);
path.pop();
```

Heaps / Top-K



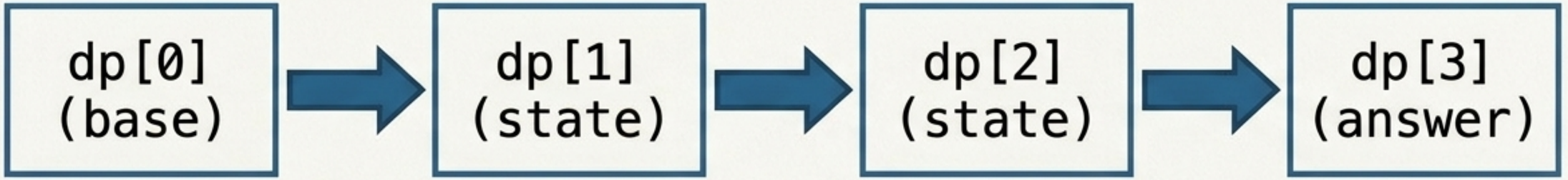
Triggers: Repeated min/max; Top K; merge sorted streams; scheduling; streaming median.

Mental Model: Fast access to smallest/largest element while retaining only the candidates you need.

JS Warning: Know that `push` = bubble up, `pop` = move root + bubble down.

Gemini Notebook

Dynamic Programming



Recognition: Repeated subproblems + optimal/counting decision.

The Diagnostic Questions:

1. What does `dp[i]` or `dp[i][j]` exactly mean?
2. What transition gets me here?
3. What are the base cases?

Last-Minute Core: Climbing Stairs, House Robber, Maximum Subarray.

Core Philosophy: Understand the state + transition rather than memorizing code. Start with the smallest state.

Gemini Notebook

0–5 min (Clarify):	5–10 min (Plan):	10–30 min (Code):	30–38 min (Test):	38–45 min (Follow-up):
Restate problem; examples; constraints; edge cases.	Brute force → identify pattern → optimized approach → complexity.	Keep talking; clear variables; don't over-engineer.	Normal + edge + empty/singleton/ boundary/cycle cases.	Complexity, optimization, alternatives.

Pair/sorted → two pointers • contiguous → sliding window • next greater/smaller → monotonic stack • ordered/monotonic answer → binary search • level/shortest unweighted → BFS • explore/reachability → DFS • dependencies → topo sort • repeated min/max → heap • all choices → backtracking • overlapping optimal subproblems → DP • dynamic components → union-find • prefixes → trie

Recognition Mantra: Spot the pattern → Identify the data structure → Apply the algorithm → Handle constraints & edges

Gemini Notebook